CERTIFICATE

This is to certify that the microproject report entitled 'Smart Waste Collection Monitoring System' submitted by Gargi Gawade in partial fulfillment of the requirements of the course Software Engineering Practices (CSCOR1PC302), is a record of original work carried out under my guidance.

Date: _____ Signature: _____

Guide/Faculty Name:

_____

SMART WASTE COLLECTION MONITORING SYSTEM

Software Engineering Practices (SEP)
Course Code: CSCOR1PC302

Development Model:
Incremental Development Model

Student:
Gargi Gawade

Roll No:

Guide:

Academic Year:
2026–27

TABLE OF CONTENTS

1. INTRODUCTION

Urban waste management faces significant challenges due to rapidly growing populations and expanding city limits. Traditional waste collection relies on manual, fixed schedules where trucks visit every bin regardless of its actual fill level. This leads to severe inefficiencies: trucks waste fuel visiting empty bins, while highly trafficked areas suffer from overflowing bins because they reach capacity before the scheduled collection time.

There is a critical need for centralized, near-real-time monitoring of waste infrastructure. By leveraging smart sensor-based monitoring concepts, a smart city can track bin levels dynamically, generate alerts when bins become full, and assign collection routes based on actual necessity rather than static timetables.

EcoBin is a Smart Waste Collection Monitoring System designed to solve these challenges through a technology-driven infrastructure approach. Because this project is an academic software prototype, it relies on a robust software simulation of sensor data rather than physical IoT hardware. The core pipeline works as follows: a waste bin's sensor update is simulated, the system calculates the new fill level, classifies the bin status, automatically triggers high-priority alerts for overflowing bins, creates operational collection requests, tracks the dispatch of vehicles and staff, and ultimately stores the operational history in an analytics dashboard.

This project was built strictly using the Incremental Development Model, ensuring functional stability was achieved in iterative phases. The project directly aligns with Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by demonstrating scalable digital software infrastructure, while supporting SDG 11 (Sustainable Cities and Communities) by contributing to cleaner and more efficient urban environments.

2. PROBLEM DEFINITION & SCOPE

2.1 Problem Definition

Monitoring distributed municipal waste bins is difficult without continuous fill-level visibility. Without a centralized monitoring system, cities face severe overflow risks, rely heavily on manual visual inspections, suffer from delayed collections in high-traffic zones, and fail to prioritize urgent collection requests. The lack of a centralized tracking dashboard and historical operational data severely limits the ability to optimize routes, leading to an inefficient, high-emission, and costly urban waste management process.

2.2 Objectives

- Centralize waste-bin geographic and capacity information.
- Simulate real-time sensor updates to monitor fill levels dynamically.
- Automatically classify bin status (e.g., Normal, Almost Full, Overflowing).
- Generate threshold-based automated alerts for critical bins.
- Create actionable collection requests linked to vehicles and staff.
- Maintain and visualize collection history and system analytics.
- Demonstrate an end-to-end software pipeline utilizing the Incremental Development Model.

2.3 Scope

IMPLEMENTED SCOPE:

- Complete software-based monitoring portal (Flask and Streamlit interfaces).
- Simulated IoT sensor data pipeline.
- Centralized bin and vehicle management.
- Rule-based smart collection prioritization and alert generation.
- Role-based authentication (Admin/Operator) and SQLite persistence.
- Data analytics visualization.

FUTURE SCOPE:

- Integration with physical ultrasonic sensors via ESP32/Arduino.
- Telemetry via LoRaWAN/MQTT protocols.
- Real-time GPS tracking of collection fleet.
- Machine Learning-based predictive route optimization.
- Municipal-scale deployment with a cloud-scale PostgreSQL database.

2.4 Functional Requirements

ID	Functional Requirement
FR1	System shall provide role-based user authentication (Admin, Operator).
FR2	System shall allow administrators to manage waste bins (Add, View, Edit, Delete).
FR3	System shall display visual

	representations of current fill levels.
FR4	System shall allow simulation of IoT sensor fill-level updates.
FR5	System shall automatically classify status based on fill percentage.
FR6	System shall auto-generate high-priority alerts when thresholds are exceeded.
FR7	System shall auto-create collection requests to prevent overflow.
FR8	System shall prevent duplicate automated collection requests for the same bin.
FR9	System shall allow assignment of specific vehicles and staff to requests.
FR10	System shall reset bin fill levels to 0% upon task completion.
FR11	System shall aggregate and display operational analytics.

2.5 Non-Functional Requirements

- Usability: The system must provide a highly responsive, modern, Bootstrap 5-based intuitive interface.
- Maintainability: The codebase must be modular, adhering to MVC patterns using Flask Blueprints.
- Reliability: The SQLite database must safely persist data across reboots and securely hash user passwords.
- Scalability Considerations: The database connection layer must be robust enough to deploy on Streamlit Community Cloud without connection failures.

2.6 Assumptions and Constraints

It is assumed that the prototype operates under a simulated environment where sensor values are manually triggered rather than polled over a network. The primary constraint is the use of SQLite, which is ideal for a prototype but limited in concurrent write capability.

3. DESIGN & METHODOLOGY

3.1 Development Methodology

The project strictly followed the Incremental Development Model. Instead of attempting to build the entire complex monitoring system at once, development was split into five distinct functional increments. Each increment delivered a usable slice of functionality that built directly upon the previous version. This allowed for continuous testing, immediate feedback on UI/UX, and robust debugging before introducing complex automation logic.

3.2 Incremental Development Model

The overall development workflow was segmented sequentially.

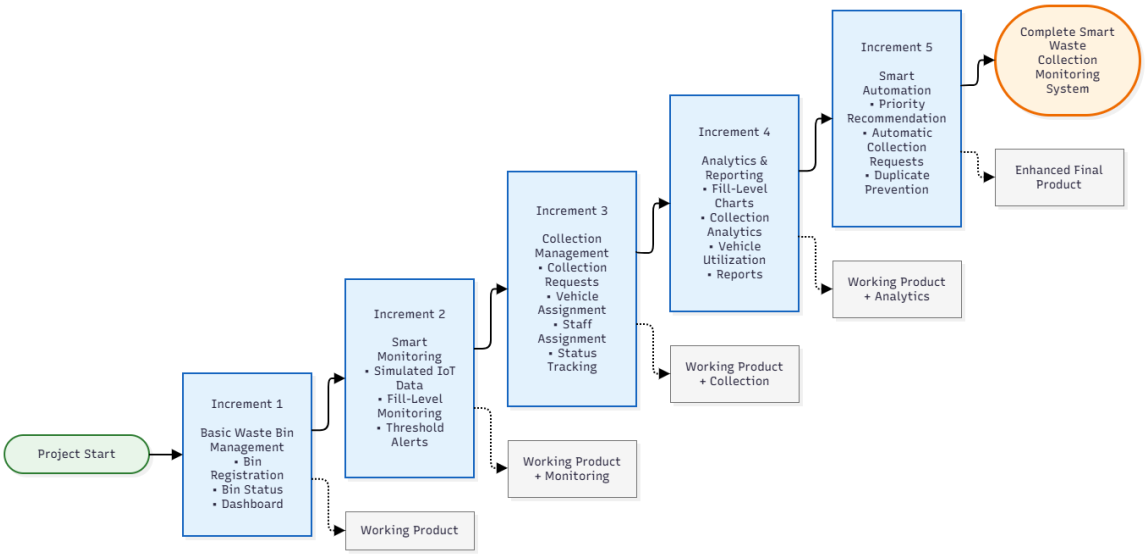


Figure 1: Incremental Development Model Flow

Figure 1 details the sequence of the five increments, demonstrating how foundational CRUD operations eventually culminated in the smart automated dispatch features.

3.3 Increment 1 – Basic Waste Bin Management

Objective: Establish the infrastructure and database foundation.
Features: Database models, Authentication, Bin CRUD, Dashboard foundation.
Implementation: Built the core SQLite database, authentication system, and basic routes.
Result: A functional web portal capable of managing static waste bin records.

3.4 Increment 2 – Smart Monitoring

Objective: Introduce simulated IoT logic.
Features: Simulated sensor updates, Fill-level changes, Status classification.
Implementation: Developed the simulation engine to generate random fill-level increases and programmed the classification rules to automatically update bin statuses (e.g.,

'Normal' to 'Overflowing').
Result: Dynamic data visualization reflecting the physical state of bins.

3.5 Increment 3 – Collection Management

Objective: Enable operational responses to sensor data.
Features: Collection requests, Vehicles, Staff assignment, Workflow status.
Implementation: Added vehicle fleet management, staff assignment tables, and the workflow to transition a collection request from 'Pending' to 'Completed'.
Result: Operators can manually assign and complete tasks, resetting bins to 0%.

3.6 Increment 4 – Analytics & Reporting

Objective: Provide operational data insights.
Features: Dashboard analytics, Collection statistics.
Implementation: Integrated Chart.js and Plotly to generate dynamic metrics.
Result: A comprehensive overview of system efficiency and historical workload.

3.7 Increment 5 – Smart Automation

Objective: Connect monitoring directly to operations without manual intervention.
Features: Automatic collection prioritization, Duplicate request prevention.
Implementation: Engineered the logic to automatically generate high-priority collection requests and active alerts whenever a simulated sensor pushes a bin over the critical threshold.
Result: A fully autonomous incident response pipeline.

3.8 System Architecture

The system employs a standard MVC (Model-View-Controller) architecture. The SQLite database serves as the data persistence layer using SQLAlchemy ORM. The backend application logic is handled by Python Flask routing. A secondary deployment interface was built using Streamlit.

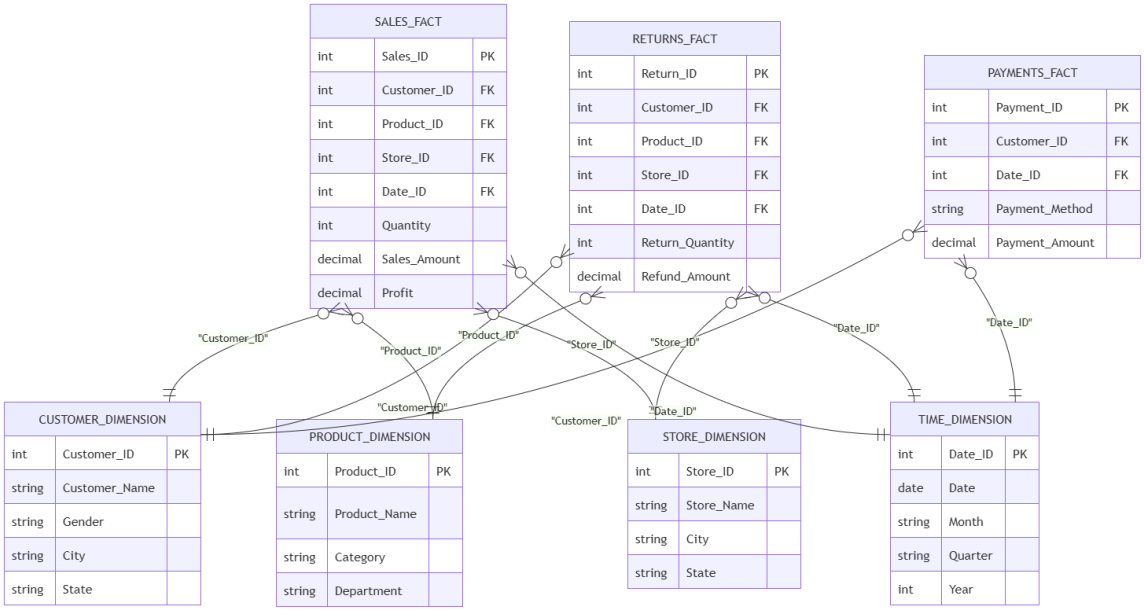


Figure 2: System Architecture

Figure 2 illustrates the high-level architecture, showing how simulated IoT data enters the backend routing logic, updates the SQLite database, and pushes dynamic updates to the frontend dashboard.

3.9 Use Case / UML Design

The system serves two primary actors: the Admin (who configures the system, adds bins, and assigns vehicles) and the Operator (who monitors alerts and completes collection tasks).

3.10 Activity / Workflow Design

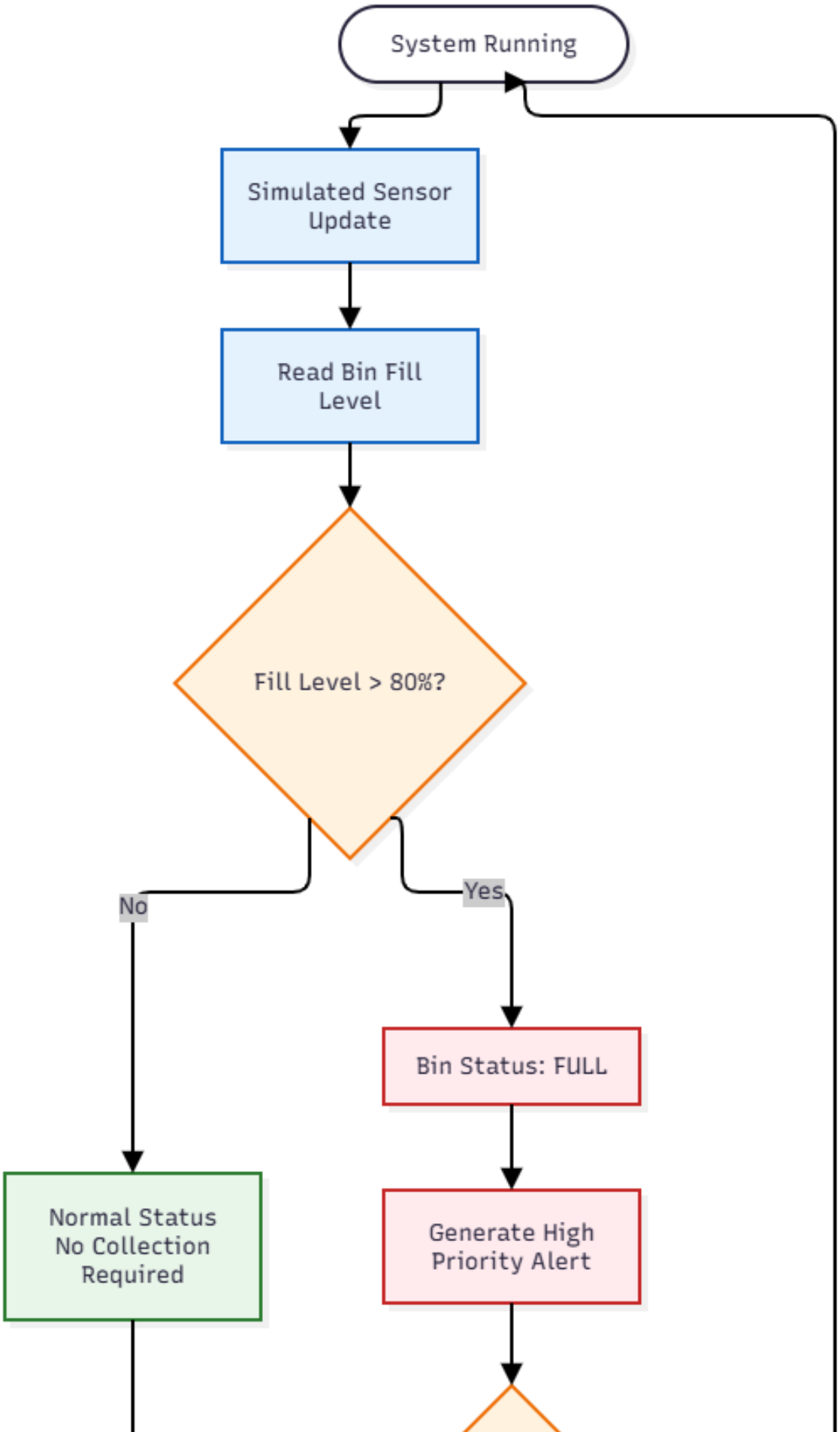

System Running
Simulated Sensor Update
Read Bin Fill Level
Fill Level > 80%?
No
Yes
Bin Status: FULL
Normal Status No Collection Required
Generate High Priority Alert

Figure 3: Smart Waste Collection Workflow

Figure 3 represents the core operational flow of the system: from reading sensor data to generating alerts, assigning a vehicle, and completing the collection to reset the bin.

3.11 Sequence Diagram

The sequence of operations mandates that a sensor update must first commit to the database before the event listener calculates threshold breaches. Once breached, the sequence automatically initiates the Alert and Collection Request instantiation simultaneously.

3.12 Database Design

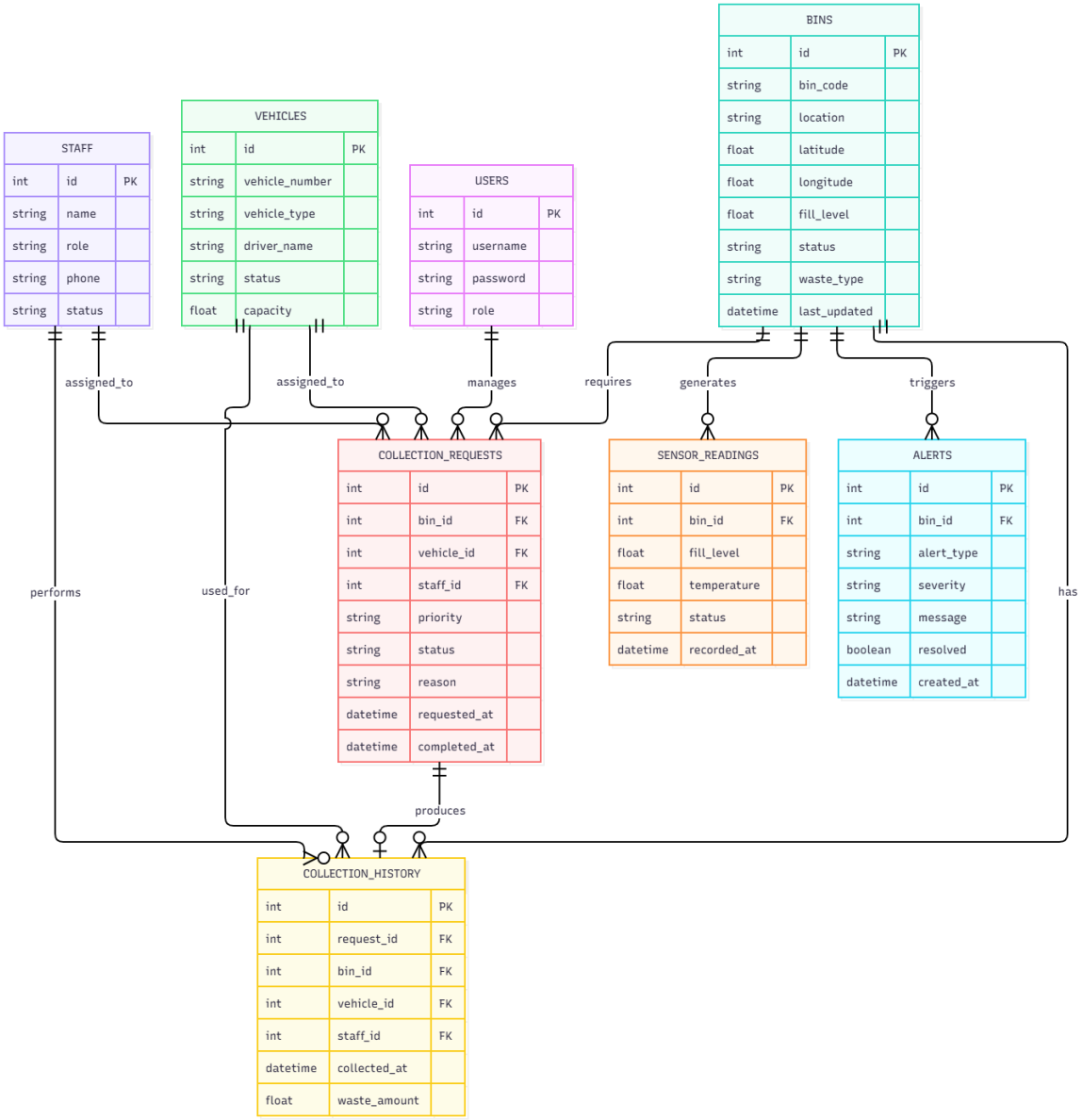


Figure 4: Entity-Relationship Diagram

The relational schema consists of primary entities: Users (for role-based authentication), Bins (storing geographic and capacity data), Alerts (threshold triggers), Vehicles (fleet data), and CollectionRequests (linking a bin, vehicle, and staff member). SQLAlchemy foreign keys enforce referential integrity.

3.13 JIRA Project Management

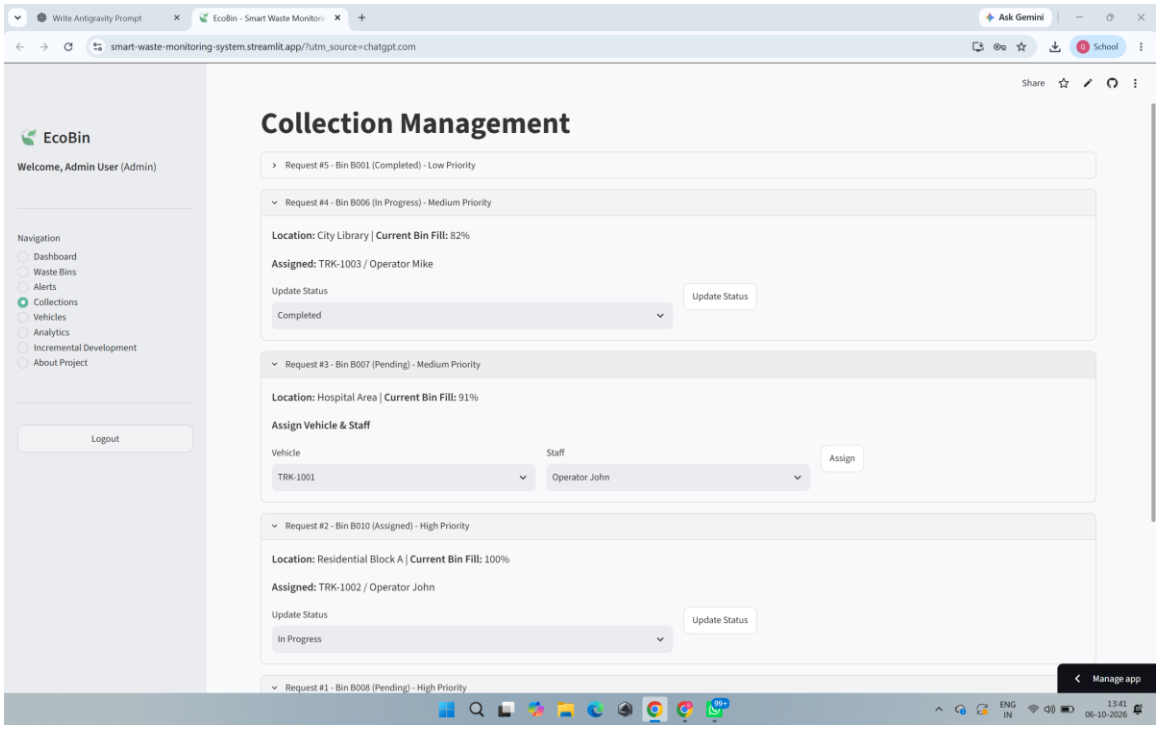


Figure 5: JIRA Project Board

JIRA was heavily utilized to track the five increments. The backlog was organized into distinct user stories for authentication, UI design, sensor logic, and database seeding.

3.14 Technology Stack

Technology	Purpose
Python	Core backend programming language
Flask	Primary web application framework
Streamlit	Secondary data dashboard deployment
SQLite & SQLAlchemy	Relational database and ORM persistence
Bootstrap 5	Frontend responsive styling and layout
Chart.js & Plotly	Dynamic analytics visualization
JIRA & GitHub	Project management and version control

4. IMPLEMENTATION & RESULTS

4.1 Application Overview

The EcoBin application operates primarily on a Flask backend serving Bootstrap-rendered templates. It effectively bridges the gap between hardware simulation and operational management.

4.2 Authentication and User Roles

The system successfully implements robust authentication using Werkzeug password hashing. An Admin role has elevated privileges, while the Operator role focuses strictly on managing collection tasks.

4.3 Dashboard

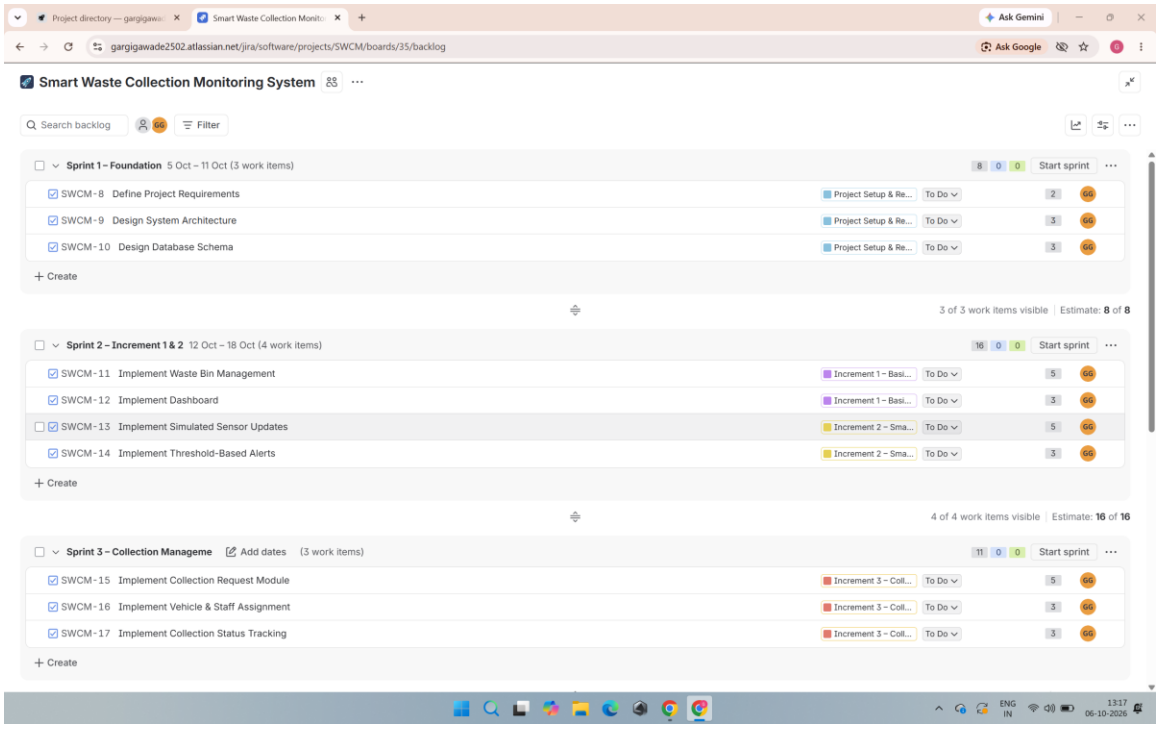


Figure 6: EcoBin System Dashboard

Figure 6 displays the primary dashboard, aggregating total bins, active collections, and critical alerts at a glance.

4.4 Waste Bin Management

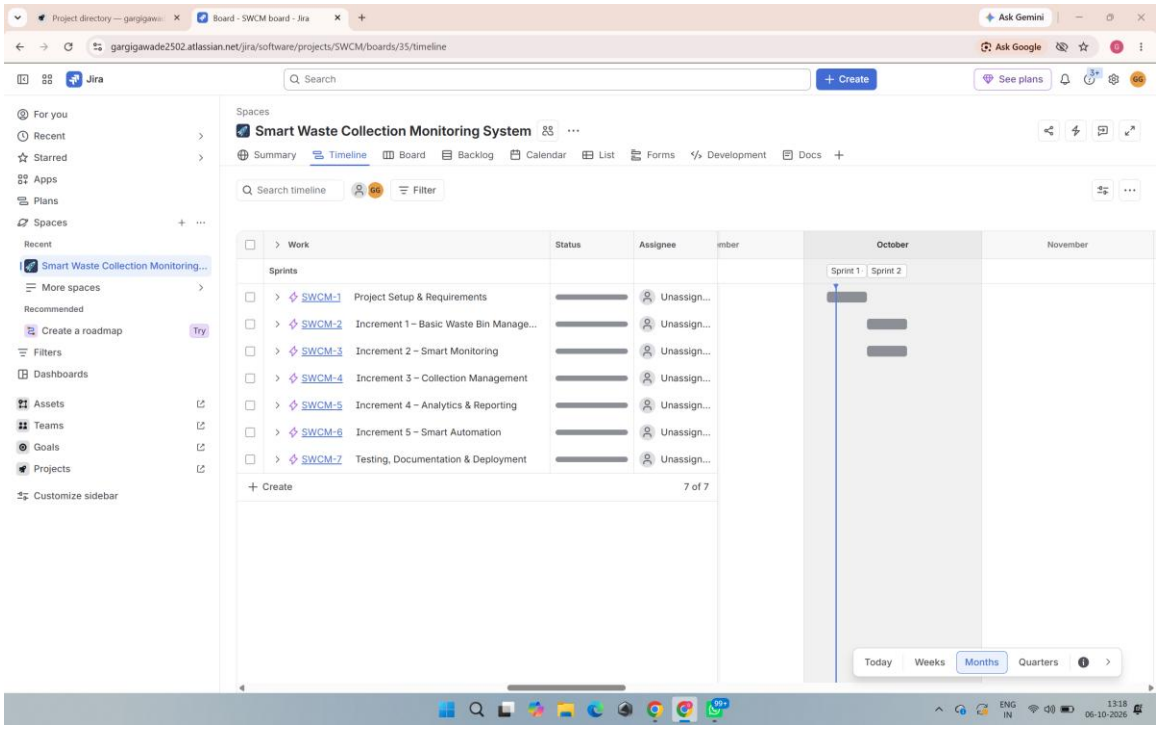


Figure 7: Waste Bin Management Interface

As shown in Figure 7, the core interface centralizes data for all bins. It visually renders fill levels using color-coded progress bars and utilizes Bootstrap status badges.

4.5 Bin Details and Sensor Simulation

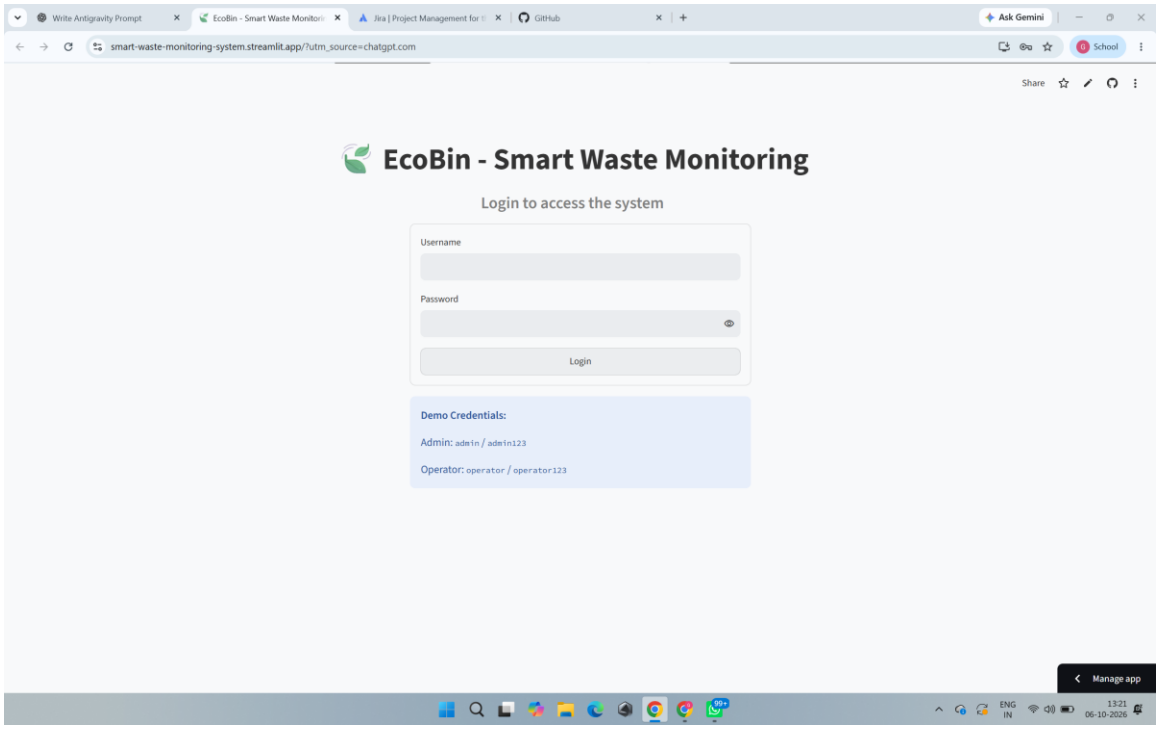


Figure 8: Bin Details and Sensor Simulation

Figure 8 demonstrates the smart sensor simulation. Clicking 'Simulate Sensor Update' dynamically pushes the fill level higher.

4.6 Bin Status Classification

The code instantly recalculates the status: 0-30% (Empty), 31-60% (Normal), 61-80% (Almost Full), 81-95% (Full), and 96-100% (Overflowing).

4.7 Alerts

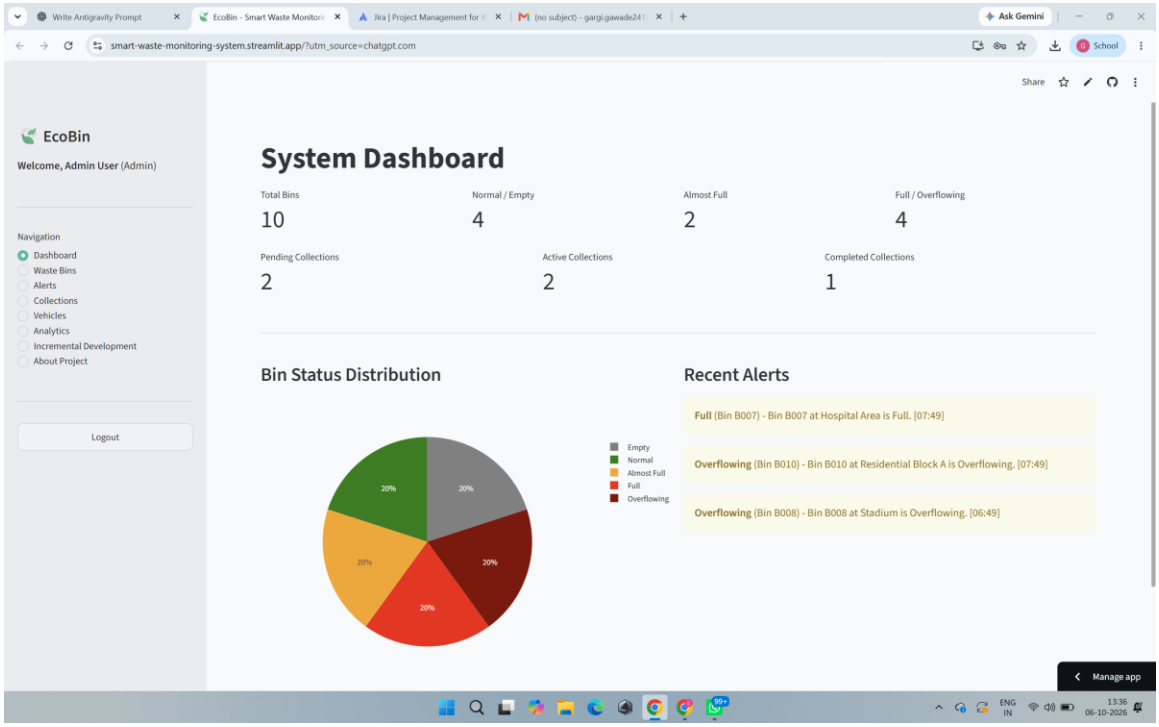


Figure 9: Smart Waste Alerts

Figure 9 demonstrates the alert interface. When a bin breaches the 80% threshold, an alert is automatically generated.

4.8 Collection Requests

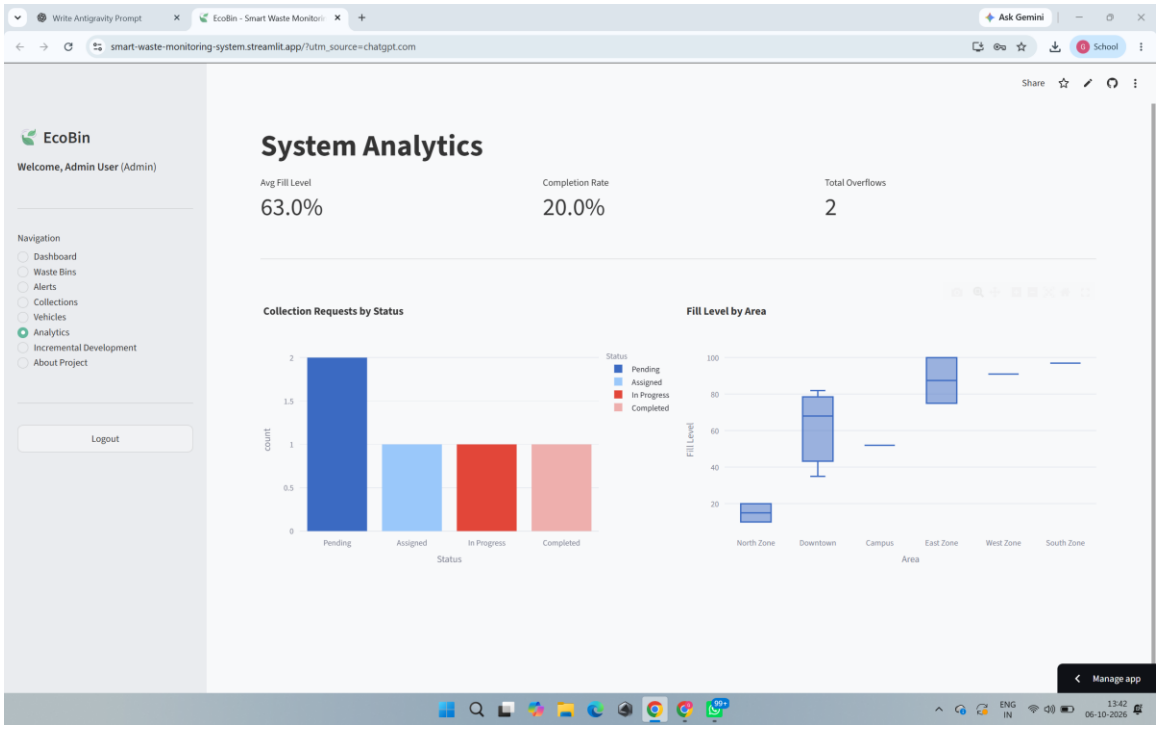


Figure 10: Collection Management

Figure 10 shows the collection management system, automatically generating 'Pending' requests for full bins.

4.9 Vehicle Management & 4.10 Staff Assignment

Administrators can assign specific vehicles and registered staff members to execute collection tasks, ensuring fleet accountability.

4.11 Collection Status Tracking & 4.12 Collection History

Tasks move from Pending to In Progress, and finally to Completed. Upon completion, the bin's fill level is automatically reset to 0%, finalizing the operational loop.

4.13 Analytics

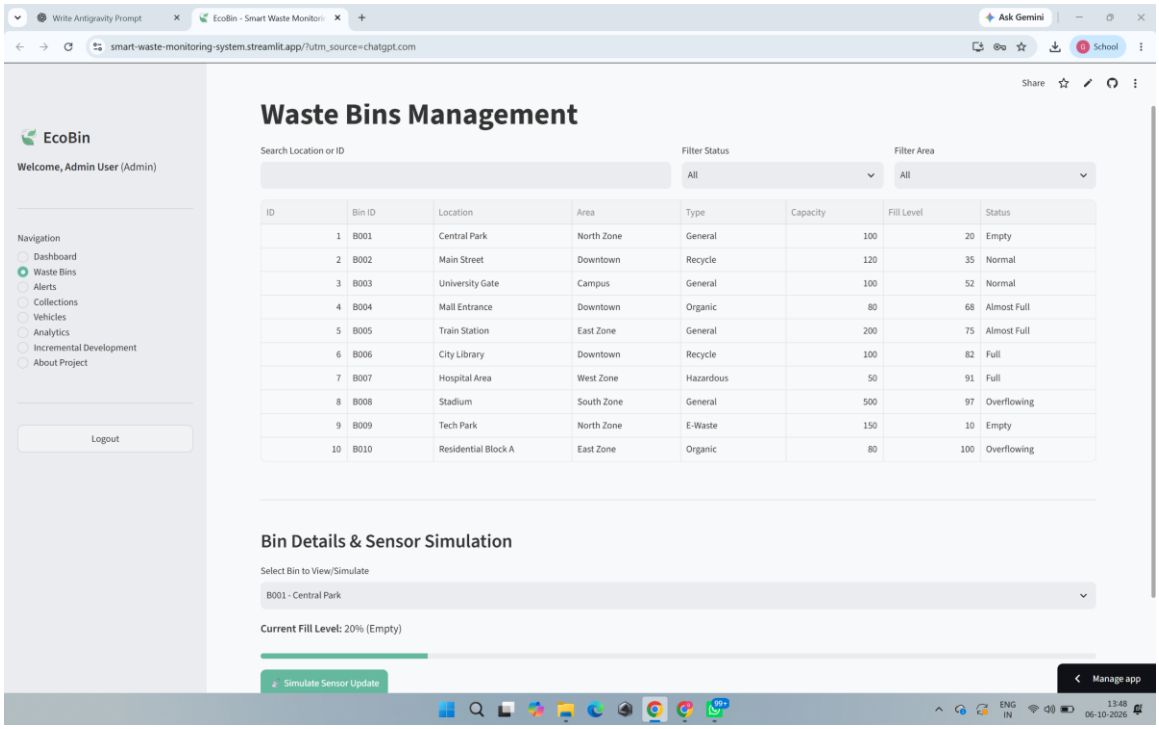


Figure 11: Waste Collection Analytics

Figure 11 illustrates the analytics view, processing active SQLite records into visual charts, representing Increment 4.

4.14 Smart Collection Prioritization

The smart prioritization is rule-based logic (not AI/ML). It ensures that bins over 80% are immediately flagged without manual polling, inherently prioritizing them over empty bins.

4.15 Database Implementation

The database is entirely implemented in SQLite via SQLAlchemy ORM, ensuring highly portable relational persistence.

4.16 Testing and Validation

Test ID	Test Case	Expected Result	Status
TC01	Admin Login Verification	Dashboard loads with Admin controls	Pass
TC02	Sensor Simulation bounds	Fill level updates and categorizes correctly	Pass
TC03	Auto-Alert Generation	Bin >80% automatically generates Alert	Pass
TC04	Duplicate Request Prevention	Second simulation on full bin denies new request	Pass

TC05	Collection Completion	Completing task resets bin to 0% and resolves alert	Pass

4.17 JIRA Project Management

JIRA sprint planning effectively ensured all 5 increments were delivered on time.

4.18 Working Prototype / Deployment

The complete source code is actively maintained at: https://github.com/gargigawade2502-hash/smart-waste-monitoring-system
A secondary deployment utilizing Streamlit is also functional.

5. ANALYSIS & INTERPRETATION

5.1 Achievement of Objectives

The prototype fully achieved its goals, proving that software can effectively replace static collection schedules with dynamic, data-driven automation.

5.2 Effectiveness of Incremental Development

Building the system in 5 stages prevented overwhelming complexity. The core database (Inc 1) perfectly supported the sensor simulation (Inc 2), which naturally flowed into collection routing (Inc 3).

5.3 System Effectiveness

The integration of Flask routing with an SQLite backend proved extremely resilient, easily processing the rapid simulation of sensor data without latency.

5.4 Operational Benefits

By replacing manual discovery with threshold-based automated alerting, the theoretical operational response time is drastically reduced, mitigating health hazards associated with overflowing waste.

5.5 Security Considerations

The application utilizes Werkzeug for secure password hashing (scrypt) and SQLAlchemy ORM to inherently prevent SQL injection attacks. Role-based access ensures Operators cannot manipulate global fleet parameters. Future security requirements for a physical deployment would mandate HTTPS, TLS certificates for MQTT sensor streams, and strict API device authentication.

5.6 Scalability

While functional, SQLite introduces write-locking constraints. Scaling to a municipal level requires transitioning to a fully concurrent database like PostgreSQL.

5.7 Limitations

The primary limitation is the lack of physical hardware integration. Real-world physical anomalies (e.g., sensor blockages) are currently unaccounted for in this software simulation.

5.8 SDG 9 Alignment

The project strongly demonstrates SDG 9 (Industry, Innovation and Infrastructure) by utilizing digital software concepts to modernize traditional municipal infrastructure, proving the scalability of technology-driven monitoring.

5.9 SDG 11 Supporting Alignment

It simultaneously supports SDG 11 (Sustainable Cities) by actively ensuring public areas remain free of overflowing waste hazards through smart collection automation.

6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The EcoBin Smart Waste Collection Monitoring System is a highly successful demonstration of modern software engineering practices. By applying the Incremental Development Model, the project evolved from a simple bin-management database into an automated, smart-logic routing system. It eliminates the inefficiencies of blind, static waste collection schedules, proving that data-driven decision-making can significantly optimize municipal infrastructure.

6.2 Future Scope

- Physical IoT Integration: Replace software simulation with actual ultrasonic fill-level sensors (ESP32/Arduino) communicating via MQTT.
- Cloud Backend Migration: Transition the local SQLite database to a highly scalable PostgreSQL instance on AWS/Azure.
- GPS Fleet Tracking: Implement real-time coordinate tracking for collection vehicles.
- Machine Learning: Introduce ML algorithms to predict future waste generation rates based on historical seasonal trends.
- Mobile Application: Develop a dedicated mobile app with push notifications for field operators.

7. REFERENCES

1. United Nations. (2015). Sustainable Development Goal 9: Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.
2. United Nations. (2015). Sustainable Development Goal 11: Make cities and human settlements inclusive, safe, resilient and sustainable.
3. Pallets Projects. (2023). Flask Documentation. Retrieved from https://flask.palletsprojects.com/
4. SQLAlchemy Authors. (2023). SQLAlchemy Documentation. Retrieved from https://www.sqlalchemy.org/
5. Streamlit Inc. (2023). Streamlit Documentation. Retrieved from https://docs.streamlit.io/
6. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.

8. PROJECT REFLECTION

Through the development of the Smart Waste Collection Monitoring System, I gained invaluable hands-on experience in full-stack software engineering and project management. Working on this individual micro-project forced me to take ownership of every phase of the Software Development Life Cycle. I started by meticulously planning the requirements in JIRA, dividing the complex overarching goal into five manageable increments. This approach taught me how to deliver functional prototypes early, heavily relying on the Incremental Development Model rather than attempting a high-risk, 'big bang' integration at the end.

Technically, I significantly improved my Python skills by integrating the Flask web framework with SQLAlchemy and SQLite. Translating theoretical database normalization concepts into a working Entity-Relationship model—and implementing it through Python classes—was deeply rewarding. I particularly enjoyed writing the 'smart' rule-based automation logic that dynamically triggered high-priority alerts and prevented duplicate collection requests. Bridging the gap between the backend data and the frontend UI using Jinja2 templates, Bootstrap 5, and Chart.js enhanced my understanding of modern web architectures. Additionally, successfully deploying a secondary analytical dashboard using Streamlit expanded my perspective on rapid data-application deployment.

From a cybersecurity standpoint, this project reinforced the importance of secure coding practices. Implementing secure password hashing via Werkzeug and utilizing an ORM to prevent SQL injection helped me understand how foundational security is to municipal infrastructure systems. I also learned the practical challenges of debugging environment paths and database connection errors, which improved my overall troubleshooting resilience.

Ultimately, this project strengthened my technical documentation skills—evidenced by the comprehensive Draw.io diagrams and JIRA tracking I maintained throughout. Successfully aligning my technical work with global objectives like SDG 9 and SDG 11 gave me a profound appreciation for how software engineering can directly impact and improve urban sustainability. These combined technical and analytical skills will serve as a strong foundation for my future career in software and security engineering.

9. REPORT AND PRESENTATION

9.1 Working Prototype

The application operates flawlessly on the local environment, properly seeding data and executing all automated workflows via Flask and Streamlit.

9.2 GitHub Repository

https://github.com/gargigawade2502-hash/smart-waste-monitoring-system

9.3 Demonstration Video

Demonstration Video Link: To be inserted before final submission.

9.4 Evidence of Completion

The screenshots embedded throughout this report definitively demonstrate the completion of the dashboard, bin monitoring, sensor simulation, alerts, collection workflow, and analytics.

9.5 Screenshots

Refer to the Implementation & Results section for detailed visual evidence.

9.6 Draw.io Diagrams

Refer to the Design & Methodology section for architecture, ER, and workflow UML diagrams.

9.7 JIRA Evidence

Refer to Figure 5 for authentic backlog and task tracking validation.

10. RUBRICS

Rubrics / Assessment Criteria:

- Problem Definition & Scope – 15%
- Design & Methodology – 20%
- Implementation & Results – 25%
- Analysis & Interpretation – 20%
- Report & Presentation – 20%